EAST WEST UNIVERSITY

Assignment

Submitted to:

- *Dr. Md.Manowarul Islam*
- *Adjunct faculty*
- *Department of Computer Science and Technology*
- *East West University*

Submitted by:

- *Name : Rakib Al Hasan*
- *Student ID : 2024-1-60-143*
- *Course Title : Data Structure and Algorithm*
- *Course Code : CSE 207*
- *Section : 10*
- *Semester : Summer 2025*
- *Date of Submission : 04.08.2025*

Exercise 1

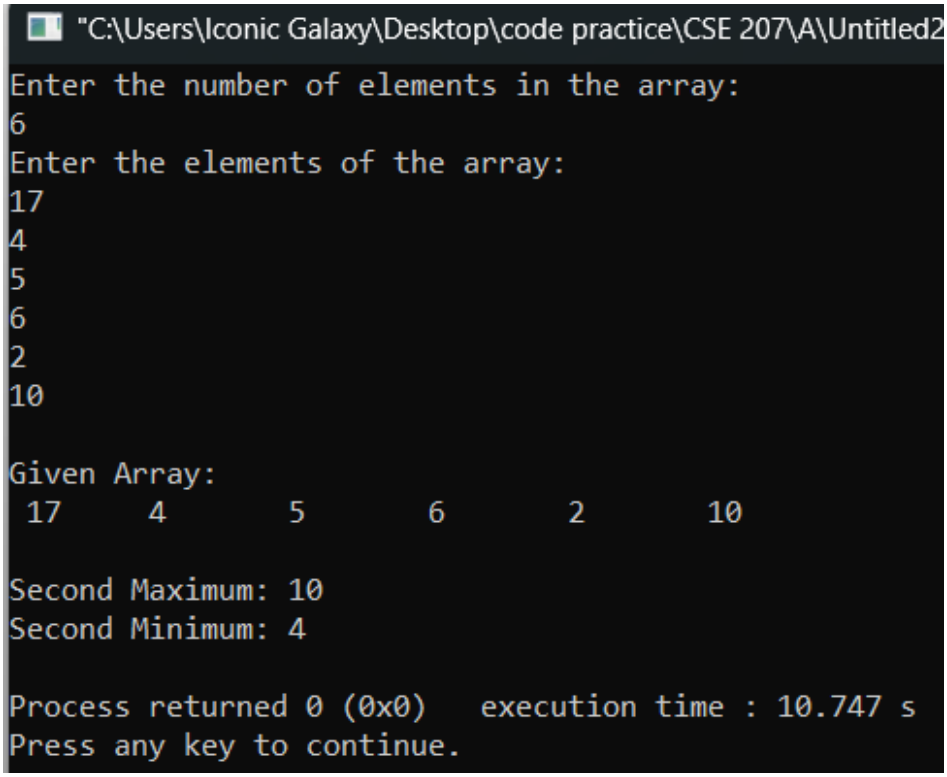
Write a program to find second maximum and minimum number of an integer array. Show the sample input and output clearly.

Sample Input	Sample Output
17, 4,5,6,2,10	Second Maximum: 10 Second Minimum: 4

```
#include <stdio.h>
#include <limits.h>

void findSecondMinMax(int arr[], int n)
{
   if (n < 2)
   {
      printf("The array must have at least two elements to find a second minimum and maximum.\n");
      return;
   }

   int max, sec_max, min, sec_min;

   if (arr[0] > arr[1])
   {
      max = arr[0];
      sec_max = arr[1];
      min = arr[1];
      sec_min = arr[0];
   }
   else
   {
      max = arr[1];
      sec_max = arr[0];
      min = arr[0];
      sec_min = arr[1];
   }
```

```
    for (int i = 2; i < n; i++)
    {
        if (arr[i] > max)
        {
            sec_max = max;
            max = arr[i];
        }
        else if (arr[i] > sec_max && arr[i] < max)
        {
            sec_max = arr[i];
        }

        if (arr[i] < min)
        {
            sec_min = min;
            min = arr[i];
        }
        else if (arr[i] < sec_min && arr[i] > min)
        {
            sec_min = arr[i];
        }
    }

    printf("\nGiven Array:\n ");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t", arr[i]);
    }
    printf("\n\nSecond Maximum: %d\n", sec_max);
    printf("Second Minimum: %d\n", sec_min);
}

int main()
{
    int n;

    printf("Enter the number of elements in the array:\n");
    scanf("%d", &n);
```

```
    int arr[n];
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    findSecondMinMax(arr, n);

    return 0;
}
```



Exercise 2

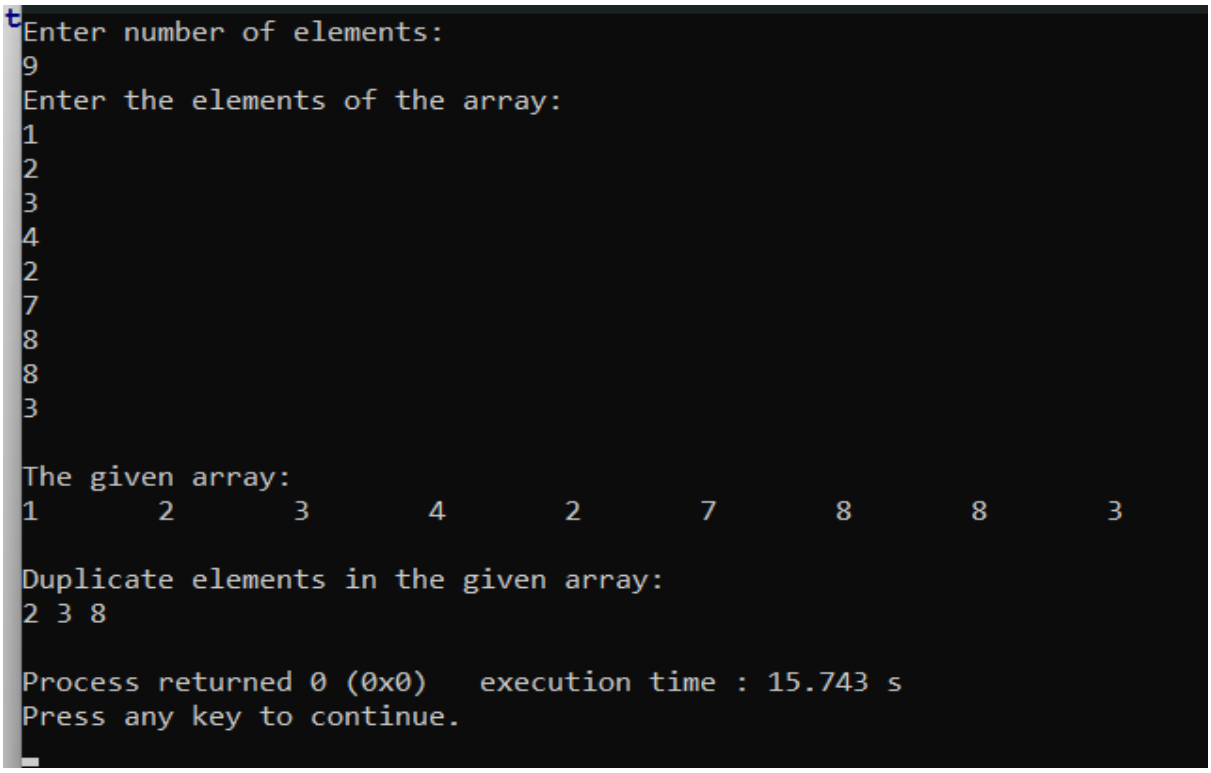
The following algorithm will print the duplicate entry in an integer array.

Write a program in C to find the duplicate number.

Sample Input	Sample Output
1, 2, 3, 4, 2, 7, 8, 8, 3	Duplicate elements in given array: 2 3 8

```
#include <stdio.h>
void findDuplicates(int arr[], int size)
{
   int duplicates[size] , dupCount = 0;
   printf("\nDuplicate elements in the given array:\n");
   for (int i = 0; i < size; i++)
   {
      for (int j = i + 1; j < size; j++)
      {
         if (arr[i] == arr[j])
         {
            int alreadyPrinted = 0;
            for (int k = 0; k < dupCount; k++)
            {
               if (duplicates[k] == arr[i])
               {
                  alreadyPrinted = 1;
                  break;
               }
            }
            if (!alreadyPrinted)
            {
               duplicates[dupCount++] = arr[i];
               printf("%d ", arr[i]);
            }
            break;
         }
      }
   }
   if (dupCount == 0)
```

```
    {
        printf("None");
    }
    printf("\n");
}

int main()
{
    int n;
    printf("Enter number of elements:\n");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("\nThe given array:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t", arr[i]);
    }
    printf("\n");
    findDuplicates(arr, n);
    return 0;
}
```


Enter number of elements:
9
Enter the elements of the array:
1
2
3
4
2
7
8
8
3
The given array:
1 2 3 4 2 7 8 8 3
Duplicate elements in the given array:
2 3 8
Process returned 0 (0x0) execution time : 15.743 s
Press any key to continue.

3. Write a program in C to delete an element at desired position from an array.

Sample Input	Sample Output
Input array elements: 10 20 30 40 50 Input position to delete: 2	Array elements: 10, 30, 40, 50

```
#include <stdio.h>
void deleteElement(int arr[], int n, int position)
{

    if (position < 1 || position > n)
    {
        printf("\nInvalid position!\n");
    }
    else
    {
        for (int i = position - 1; i < n - 1; i++)
        {
            arr[i] = arr[i + 1];
        }
        (n)--;
        printf("\nArray after deletion:\n");
        for (int i = 0; i < n; i++)
        {
            printf("%d\t", arr[i]);
        }
    }
}
```

```
int main()
{
    int n, position;
    printf("Enter a positive value for n please\n");
    scanf("%d", &n);

    printf("\nValue of n is %d\n", n);
    printf("\nSize of array is %d\n", n);

    int arr[n];
    printf("\nEnter the elements of the array please\n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("\nThe given array\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t", arr[i]);
    }
    printf("\n\nEnter position to delete (starting from 1): ");
    scanf("%d", &position);
    deleteElement(arr, n, position);
    return 0;
}
```

```
Enter a positive value for n please
5

Value of n is 5

Size of array is 5

Enter the elements of the array please
10
20
30
40
50

The given array
10      20      30      40      50

Enter position to delete (starting from 1): 2

Array after deletion:
10      30      40      50
Process returned 0 (0x0)   execution time : 9.791 s
Press any key to continue.
```

4. Write a program in C to insert element in array at specified position.

Sample Input	Sample Output
Input array elements: 10, 20, 30, 40, 50 Input element to insert: 25 Input position where to insert: 3	Elements of array are: 10, 20, 25, 30, 40, 50

```
#include <stdio.h>
void insertElement(int arr[], int n, int position, int new_element)
{
   if (position < 1 || position > n + 1)
   {
      printf("\nInvalid position!\n");
      return;
   }
   for (int k = n; k >= position; k--)
   {
      arr[k] = arr[k - 1];
   }
   arr[position - 1] = new_element;
   (n)++;
   printf("\nArray after insertion:\n");
   for (int k = 0; k < n; k++)
   {
      printf("%d\t", arr[k]);
   }
}

int main()
{
   int n, position, new_element;
   printf("Enter a positive value for n please\n");
   scanf("%d", &n);

   printf("\nValue of n is %d\n", n);
   printf("\nSize of array is %d\n", n);
   int arr[n + 1];
```

```
    printf("\nEnter the elements of the array please\n");
    for (int k = 0; k < n; k++)
    {
        scanf("%d", &arr[k]);
    }

    printf("\nThe given array\n");
    for (int k = 0; k < n; k++)
    {
        printf("%d\t", arr[k]);
    }
    printf("\n\nEnter element to insert: ");
    scanf("%d", &new_element);
    printf("\nEnter position to insert (starting from 1): ");
    scanf("%d", &position);
    insertElement(arr, n, position, new_element);
    return 0;
}
```

```
Enter a positive value for n please
5

Value of n is 5

Size of array is 5

Enter the elements of the array please
10
20
30
40
50

The given array
10      20      30      40      50

Enter element to insert: 25

Enter position to insert (starting from 1): 3

Array after insertion:
10      20      25      30      40      50
Process returned 0 (0x0)   execution time : 18.489 s
Press any key to continue.
```

5. Find the elements of an array that are greater than a specific Threshold. Create a new array to store all the elements from the original array that exceed a given threshold.

Sample Input	Sample Output
Input array elements: 10,25,89,50,100 Threshold: 30	89,50,100

```
#include <stdio.h>
int filterGreaterThan(int originalArr[], int n, int newArr[], int threshold)
{
   int newCount = 0;
   for (int i = 0; i < n; i++)
   {
      if (originalArr[i] > threshold)
      {
         newArr[newCount++] = originalArr[i];
      }
   }
   return newCount;
}

int main()
{
   int n, threshold;
   printf("Enter a positive value for n please\n");
   scanf("%d", &n);

   printf("\nValue of n is %d\n", n);
   printf("\nSize of array is %d\n", n);
   int arr[n];
   printf("\nEnter the elements of the array please\n");
   for (int i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
   }
```

```
    printf("\nThe given array\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t", arr[i]);
    }
    printf("\n");
    printf("Enter threshold: ");
    scanf("%d", &threshold);

    int newArr[n];
    int newCount = filterGreaterThan(arr, n, newArr, threshold);

    printf("Elements greater than %d:\n", threshold);
    for (int i = 0; i < newCount; i++)
    {
        printf("%d", newArr[i]);
        if (i < newCount - 1)
        {
            printf(", ");
        }
    }
    printf("\n");
    return 0;
}
```

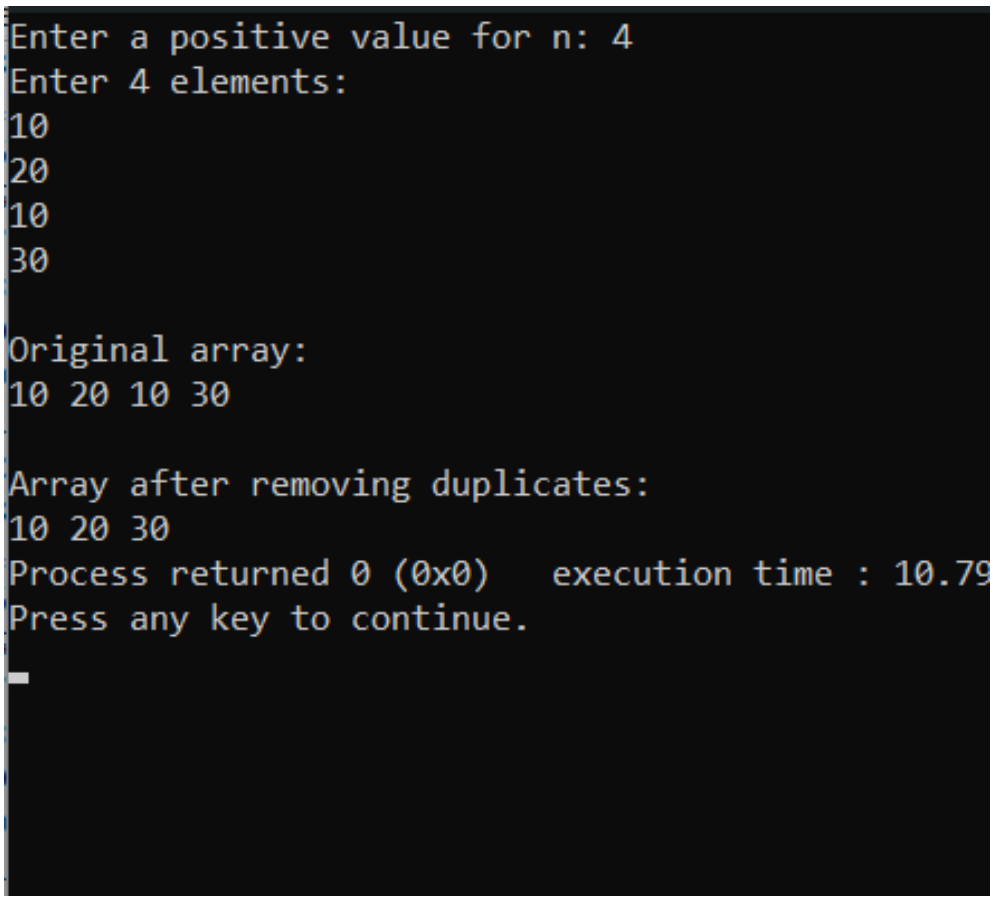
```
Enter a positive value for n please
5

Value of n is 5

Size of array is 5

Enter the elements of the array please
10
25
89
50
100

The given array
10      25      89      50      100
Enter threshold: 30
Elements greater than 30:
89, 50, 100

Process returned 0 (0x0)   execution time : 35.992 s
Press any key to continue.
```

6. Remove duplicate elements from the 1D array and print the modified array.

Sample Input	Sample Output
Input array elements: 10,20,10,30	10,20,30

```
#include <stdio.h>
int removeDuplicates(int arr[], int n)
{
   int i, j, k;
   for (i = 0; i < n; i++)
   {
      for (j = i + 1; j < n; )
      {
         if (arr[i] == arr[j])
         {
            for (k = j; k < n - 1; k++)
            {
               arr[k] = arr[k + 1];
            }
            n--;
         }
         else
         {
            j++;
         }
      }
   }
   return n;
}

int main()
{
   int n, k;
   printf("Enter a positive value for n: ");
   scanf("%d", &n);

   int arr[n];
   printf("Enter %d elements:\n", n);
   for (k = 0; k < n; k++)
```

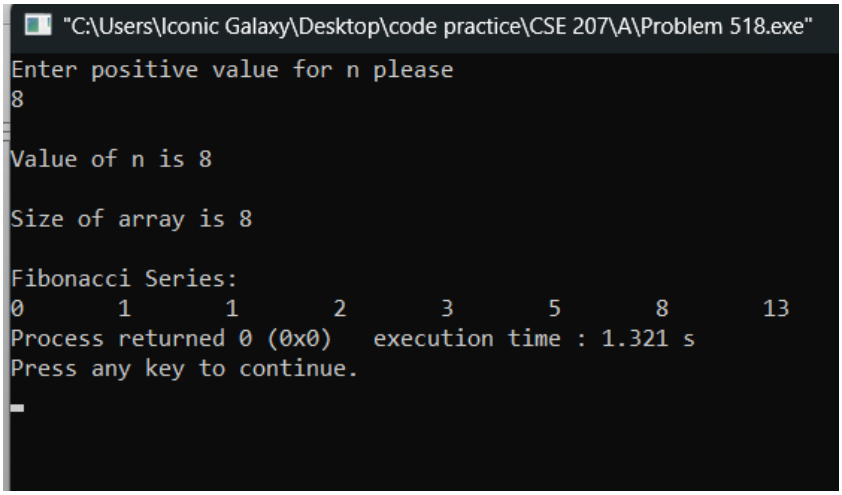
```
    {
        scanf("%d", &arr[k]);
    }

    printf("\nOriginal array:\n");
    for (k = 0; k < n; k++)
    {
        printf("%d ", arr[k]);
    }
    n = removeDuplicates(arr, n);

    printf("\n\nArray after removing duplicates:\n");
    for (k = 0; k < n; k++)
    {
        printf("%d ", arr[k]);
    }
    return 0;
}
```



7. Generate and print the first N elements of the Fibonacci series in a 1D array with initial conditions: *array* [0] =0, *array*[1]=1.

Sample Input	Sample Output
N=8.	0,1,1,2,3,5,8,13.

```
#include <stdio.h>
void generateFibonacci(int arr[], int n)
{
   if (n > 0) arr[0] = 0;
   if (n > 1) arr[1] = 1;
   for (int i = 2; i < n; i++)
   {
      arr[i] = arr[i - 1] + arr[i - 2];
   }
}

int main()
{
   int n, i;
   printf("Enter positive value for n please: ");
   scanf("%d", &n);

   if (n <= 0)
   {
      printf("n must be a positive integer.\n");
      return 1;
   }

   int arr[n];

   generateFibonacci(arr, n);
   printf("\nFibonacci Series:\n");
   for (i = 0; i < n; i++)
   {
      printf("%d\t", arr[i]);
   }
   return 0;
}
```


"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\A\Problem 518.exe"
Enter positive value for n please
8
Value of n is 8
Size of array is 8
Fibonacci Series:
0 1 1 2 3 5 8 13
Process returned 0 (0x0) execution time : 1.321 s
Press any key to continue.

8. Find if an array contains a subarray (Suppose the Subarray length is 2) with a specific pattern. The subarray pattern is defined as a sequence of two consecutive elements.

Sample Input	Sample Output
Input array elements: 1,2,**5,7,**9,10 Subarray: 5,7 Input array elements: 1,2,3,4,5 Subarray: 1,4	Found Not Found

```
#include <stdio.h>
int findSubarray(int arr[], int n, int a, int b)
{
   for (int i = 0; i < n - 1; i++)
   {
      if (arr[i] == a && arr[i + 1] == b)
      {
         return 1;
      }
   }
   return 0;
}

int main()
{
   int n, k, a, b, found;
   printf("Enter positive value for n:\n");
   scanf("%d", &n);

   int arr[n];
   printf("\nEnter %d elements:\n", n);
   for (k = 0; k < n; k++)
   {
      scanf("%d", &arr[k]);
   }
```

```
    printf("\nThe given array:\n");
    for (k = 0; k < n; k++)
    {
        printf("%d\t", arr[k]);
    }
    printf("\n\nEnter subarray (2 elements):\n");
    scanf("%d %d", &a, &b);

    found = findSubarray(arr, n, a, b);
    if (found)
    {
        printf("\nFound\n");
    }
    else
    {
        printf("\nNot Found\n");
    }
    return 0;
}
```

```
Enter positive value for n:
6

Enter 6 elements:
1
2
5
7
9
10

The given array:
1       2       5       7       9       10

Enter subarray (2 elements):
5
7

Found

Process returned 0 (0x0)   execution time : 18.487 s
Press any key to continue.
```

9. Write a program in C to modify array elements by adding 3 to elements at odd indexes and 2 to elements at even indexes.

Sample Input	Sample Output
Input array elements: 10, 20, 30,40, 50	Elements of array are: 13,22,33,42,53

```
#include <stdio.h>
void modifyArray(int arr[], int n)
{
   for (int i = 0; i < n; i++)
   {
      if (i % 2 == 0)
      {
         arr[i] += 2;
      }
      else
      {
         arr[i] += 3;
      }
   }
}

int main()
{
   int n, k;
   printf("Enter positive value for n please: ");
   scanf("%d", &n);

   printf("\nValue of n is %d\n", n);
   printf("Size of array is %d\n", n);
   int arr[n];
   printf("\nEnter the elements of the array please:\n");
   for (k = 0; k < n; k++)
   {
      scanf("%d", &arr[k]);
   }

   printf("\nThe given array:\n");
```

```
    for (k = 0; k < n; k++)
    {
        printf("%d\t", arr[k]);
    }

    modifyArray(arr, n);

    printf("\n\nModified array:\n");
    for (k = 0; k < n; k++)
    {
        printf("%d\t", arr[k]);
    }
    return 0;
}
```

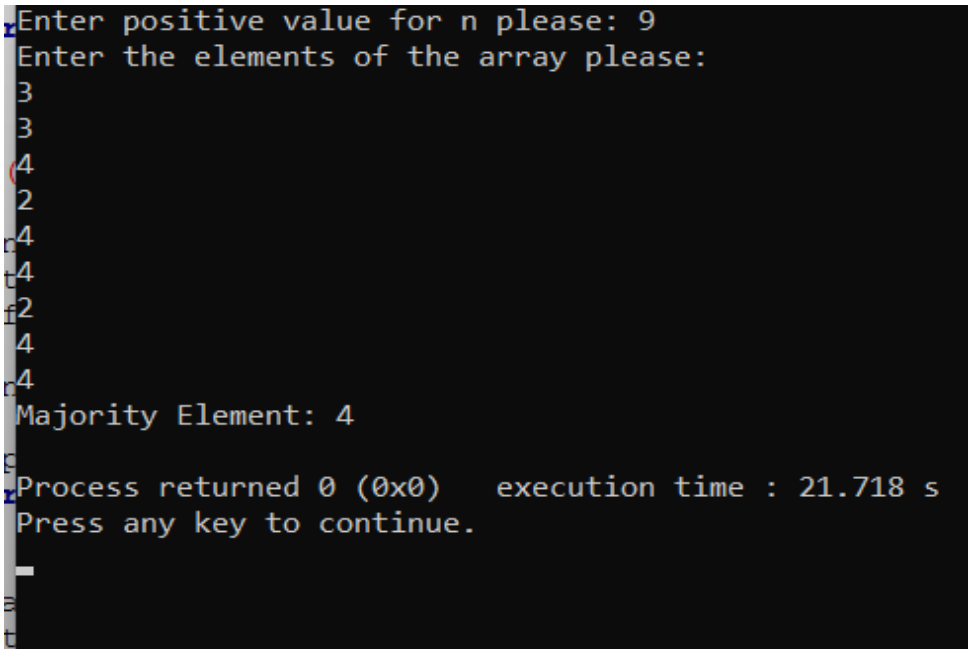
```
Enter positive value for n please: 5

Value of n is 5
Size of array is 5

Enter the elements of the array please:
10
20
30
40
50

The given array:
10      20      30      40      50

Modified array:
12      23      32      43      52
Process returned 0 (0x0)   execution time : 8.023 s
Press any key to continue.
```

10. Write a program in C to find the **majority element** in a given 1D array. A **majority element** is an element that appears **more than N/2 times**, where **N** is the size of the array. If no such element exists, print "No Majority Element Found".

Sample Input	Sample Output
Input array elements: 3, 3, 4, 2, 4, 4, 2, 4, 4	Majority Element: 4

```
#include <stdio.h>
int findMajorityElement(int arr[], int n)
{
   int i, j, count, maxCount = 0, majority = -1;

   for (i = 0; i < n; i++)
   {
      count = 1;
      for (j = i + 1; j < n; j++)
      {
         if (arr[i] == arr[j])
         {
            count++;
         }
      }
      if (count > maxCount)
      {
         maxCount = count;
         majority = arr[i];
      }
   }
   if (maxCount > n / 2)
   {
      return majority;
   }
   else
   {
      return -1;
   }
}
```

```
int main()
{
   int n, k;
   printf("Enter positive value for n please: ");
   scanf("%d", &n);

   if (n <= 0)
   {
      printf("Invalid input. Array size must be positive.\n");
      return 1;
   }
   int arr[n];
   printf("Enter the elements of the array please:\n");
   for (k = 0; k < n; k++)
   {
      scanf("%d", &arr[k]);
   }

   int majorityElement = findMajorityElement(arr, n);
   if (majorityElement != -1)
   {
      printf("Majority Element: %d\n", majorityElement);
   }
   else
   {
      printf("No Majority Element Found\n");
   }
   return 0;
}
```



11. Write a program in C to find the largest difference (max - min) between any two elements.

Sample Input	Sample Output
Input array elements: 1, 9, 3, 10	Largest difference is 1

```
#include <stdio.h>
int findLargestDifference(int arr[], int n)
{
   int i, max, min;
   max = min = arr[0];
   for (i = 1; i < n; i++)
   {
      if (arr[i] > max)
      {
         max = arr[i];
      }
      else if (arr[i] < min)
      {
         min = arr[i];
      }
   }

   return max - min;
}

int main()
{
   int n, k;

   printf("Enter positive value for n please: ");
   scanf("%d", &n);

   if (n <= 0)
   {
      printf("\nInvalid input. Array size must be positive.\n");
      return 1;
   }
```

```
    int arr[n];
    printf("\nEnter the elements of the array please:\n");
    for (k = 0; k < n; k++)
    {
       scanf("%d", &arr[k]);
    }

    printf("\nThe given array:\n");
    for (k = 0; k < n; k++)
    {
       printf("%d\t", arr[k]);
    }

    int largestDifference = findLargestDifference(arr, n);
    printf("\n\nLargest difference (max - min) between any two elements: %d\n",
largestDifference);
    return 0;
}
```

```
Enter positive value for n please: 4

Enter the elements of the array please:
1
9
3
10

The given array:
1       9       3       10

Largest difference (max - min) between any two elements: 9

Process returned 0 (0x0)   execution time : 13.168 s
Press any key to continue.
```

12. Write a program in C that moves all zeros in an array to the end while maintaining the order of non-zero elements.

Sample Input	Sample Output
Input array elements: 0, 1, 0, 3, 12	Array after shifting: 1, 3, 12, 0, 0

```
#include <stdio.h>
void moveZerosToEnd(int arr[], int result[], int n)
{
   int i, j = 0;
   for (i = 0; i < n; i++)
   {
      if (arr[i] != 0)
      {
         result[j] = arr[i];
         j++;
      }
   }
   while (j < n)
   {
      result[j] = 0;
      j++;
   }
}

int main()
{
   int n, k;

   printf("Enter positive value for n please: ");
   scanf("%d", &n);

   if (n <= 0)
   {
      printf("\nInvalid input. Array size must be positive.\n");
      return 1;
   }

   int arr[n], temp[n];
```

```
    printf("\nEnter the elements of the array please:\n");
    for (k = 0; k < n; k++)
    {
       scanf("%d", &arr[k]);
    }

    printf("\nThe given array:\n");
    for (k = 0; k < n; k++)
    {
       printf("%d\t", arr[k]);
    }
    moveZerosToEnd(arr, temp, n);
    printf("\n\nArray after moving all zeros to the end:\n");
    for (k = 0; k < n; k++)
    {
       printf("%d\t", temp[k]);
    }

    printf("\n");
    return 0;
}
```

```
Enter positive value for n please: 5

Enter the elements of the array please:
0
1
0
3
12

The given array:
0       1       0       3       12

Array after moving all zeros to the end:
1       3       12      0       0

Process returned 0 (0x0)   execution time : 13.657 s
Press any key to continue.
```

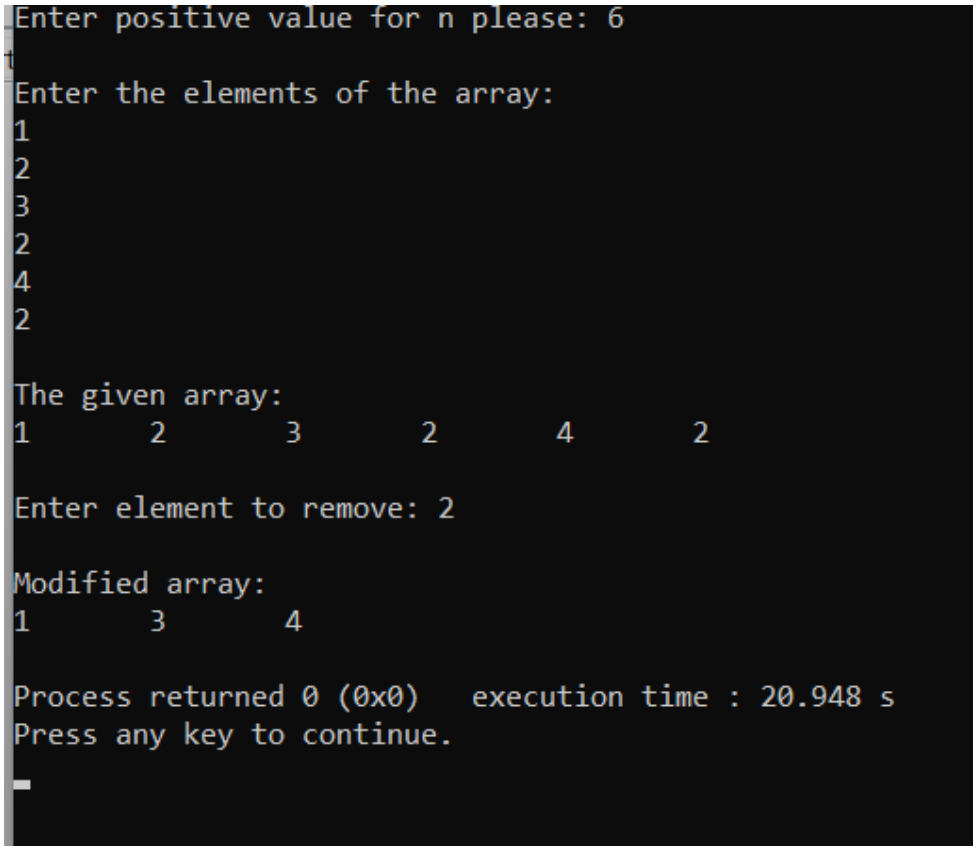
13. Write a program in C to remove all occurrences of a specific element from the array.

Sample Input	Sample Output
Input array elements: 1, 2, 3, 2, 4, 2 Element to remove: 2	Modified array: 1 3 4

```
#include <stdio.h>

int removeElement(int arr[], int n, int value)

{

   int i, j;

   for (i = 0; i < n; i++)

   {

      if (arr[i] == value)

      {

         for (j = i; j < n - 1; j++)

         {

            arr[j] = arr[j + 1];

         }

         n--;

         i--;

      }

   }

   return n;

}
```

```
int main()
{
    int n, k, value;
    printf("Enter positive value for n please: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("\nInvalid input. Array size must be positive.\n");
        return 1;
    }
    int arr[n];
    printf("\nEnter the elements of the array:\n");
    for (k = 0; k < n; k++)
    {
        scanf("%d", &arr[k]);
    }
    printf("\nThe given array:\n");
    for (k = 0; k < n; k++)
    {
        printf("%d\t", arr[k]);
    }
    printf("\n\nEnter element to remove: ");
    scanf("%d", &value);
```

```
    n = removeElement(arr, n, value);
    printf("\nModified array:\n");
    for (k = 0; k < n; k++)
    {
        printf("%d\t", arr[k]);
    }
    printf("\n");
    return 0;
}
```



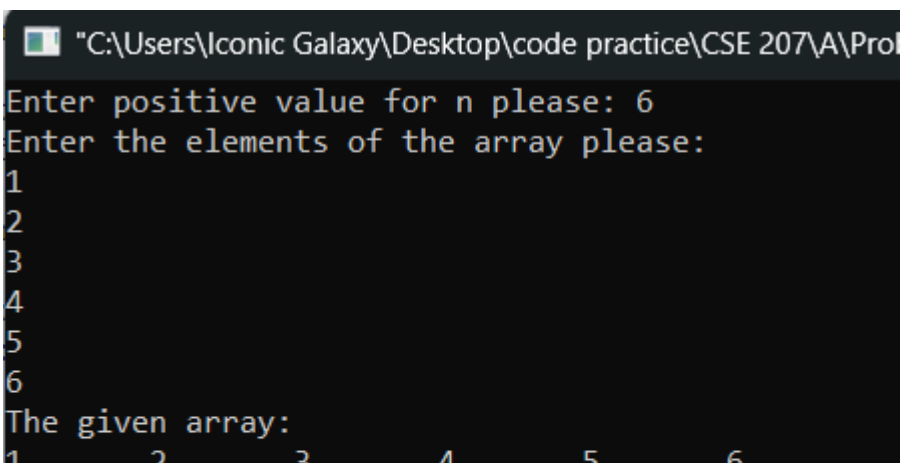
14. Write a program in C to rearrange the elements so that the first element is the largest, the second is the smallest, the third is the second-largest, the fourth is the second-smallest, and so on — alternating between the next largest and the next smallest values. This rearrangement creates a "zig-zag" pattern with high and low values alternating.

Sample Input	Sample Output
Input array elements: 1, 2, 3, 4, 5, 6	Modified array: 6 1 5 2 4 3

```
#include <stdio.h>

int main()

{

   int n, k, i, j;

   printf("Enter positive value for n please: ");

   scanf("%d", &n);


   if (n <= 0)

   {

      printf("Invalid input. Array size must be positive.\n");

      return 1;

   }

   int arr[n], temp[n];
```

```
printf("Enter the elements of the array please:\n");
for (k = 0; k < n; k++)
{
   scanf("%d", &arr[k]);
}

printf("The given array:\n");
for (k = 0; k < n; k++)
{
   printf("%d\t", arr[k]);
}
printf("\n");
for (i = 0; i < n - 1; i++)
{
   for (j = i + 1; j < n; j++)
   {
      if (arr[i] > arr[j])
      {
         int t = arr[i];
         arr[i] = arr[j];
         arr[j] = t;
      }
   }
```

```
    }
    int start = 0, end = n - 1;
    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
        {
            temp[i] = arr[end--];
        }
        else
        {
            temp[i] = arr[start++];
        }
    }
    printf("Modified array:\n");
    for (i = 0; i < n; i++)
    {
        printf("%d\t", temp[i]);
    }
    printf("\n");
    return 0;
}
```

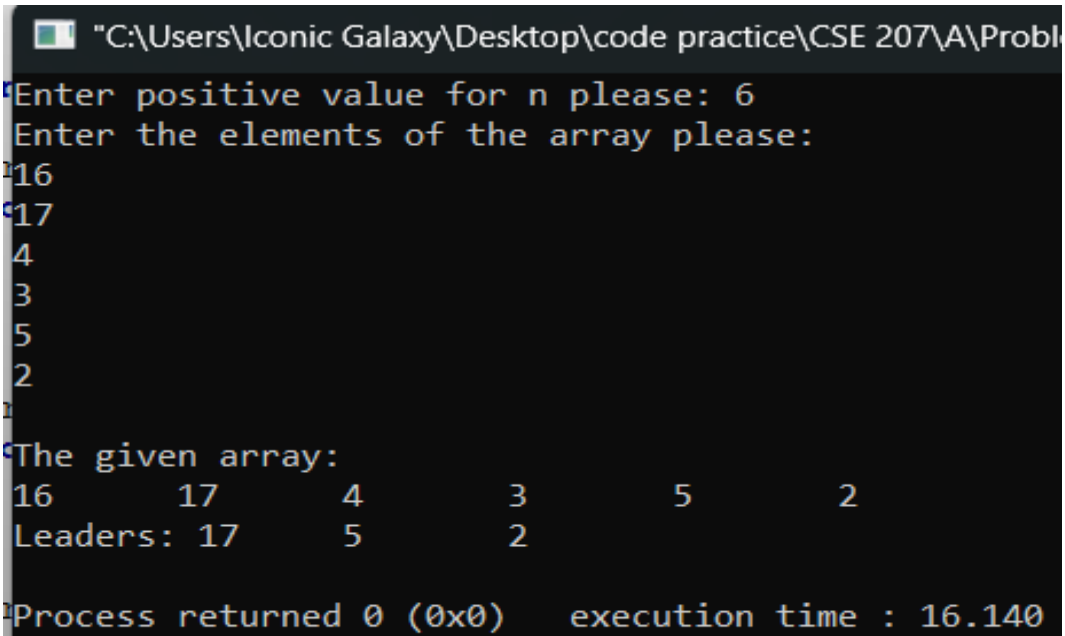


15. Write a program in C to find all leaders of an array. A leader is an element that is strictly greater than all the elements to its right. Note that the last element is always considered a leader by definition.

Sample Input	Sample Output
Input array elements: 16, 17, 4, 3, 5, 2	Leaders : 17 5 2

```
#include <stdio.h>

int main()
{
   int n, k, i, j;

   printf("Enter positive value for n please: ");
   scanf("%d", &n);

   if (n <= 0)
   {
      printf("Invalid input. Array size must be positive.\n");
      return 1;
   }

   int arr[n];

   printf("Enter the elements of the array please:\n");
   for (k = 0; k < n; k++)
      {
      scanf("%d", &arr[k]);
   }
```

```
    printf("\nThe given array:\n");
    for (k = 0; k < n; k++)
    {
        printf("%d\t", arr[k]);
    }
    printf("\n");

    printf("Leaders: ");
    for (i = 0; i < n; i++)
    {
        int isLeader = 1;
        for (j = i + 1; j < n; j++)
        {
            if (arr[i] <= arr[j])
            {
                isLeader = 0;
                break;
            }
        }
        if (isLeader)
        {
            printf("%d\t", arr[i]);
        }
    }

    printf("\n");
    return 0;
}
```



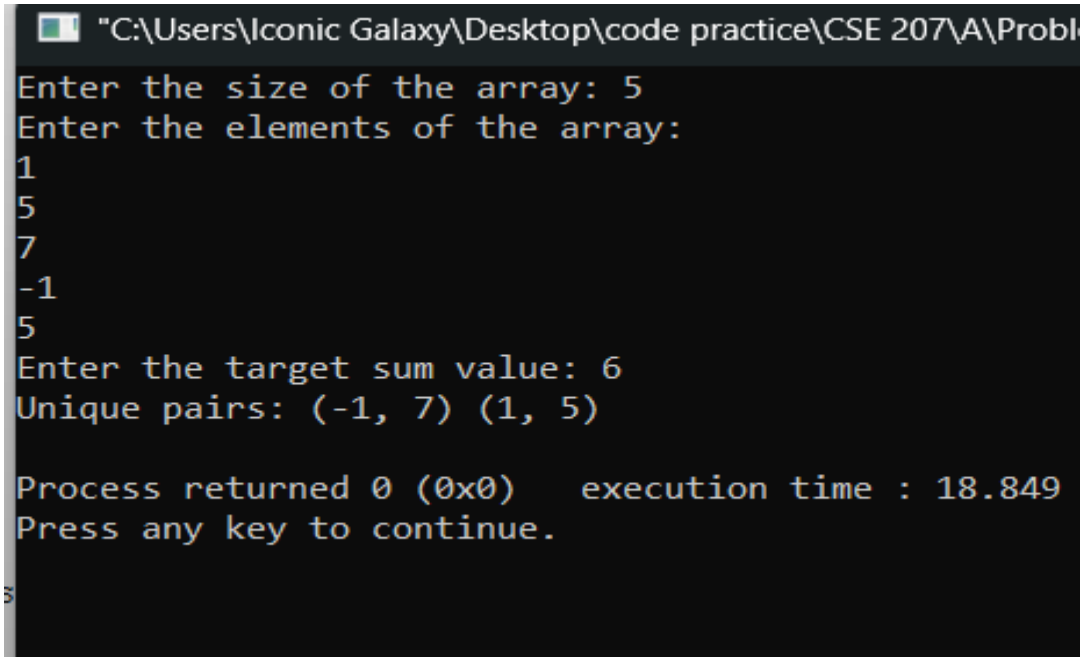
16. Write a program in C to Print all unique pairs of elements whose sum equals a given element.

Sample Input	Sample Output
Input array elements: 1, 5, 7, -1, 5 Element : 6	Pairs : (1,5) (7,-1)

```
#include <stdio.h>

void sort(int arr[], int n)
{
   for (int i = 0; i < n - 1; i++)
   {
      for (int j = 0; j < n - i - 1; j++)
      {
         if (arr[j] > arr[j + 1])
          {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
}

int main()
{
   int n, sum;

   printf("Enter the size of the array: ");
   scanf("%d", &n);
```

```
int arr[n];
printf("Enter the elements of the array:\n");
for (int i = 0; i < n; i++)
{
   scanf("%d", &arr[i]);
}

printf("Enter the target sum value: ");
scanf("%d", &sum);

sort(arr, n);

printf("Unique pairs: ");
for (int i = 0; i < n; i++)
{

   if (i > 0 && arr[i] == arr[i - 1])
   {
       continue;
   }

   for (int j = i + 1; j < n; j++)
   {
      int current_sum = arr[i] + arr[j];

      if (current_sum == sum)
      {
         printf("(%d, %d) ", arr[i], arr[j]);

         while (j + 1 < n && arr[j] == arr[j + 1])
         {
            j++;
         }
      }
      else if (current_sum > sum)
      {
         break;
      }
   }
```

```
    }

    printf("\n");
    return 0;
}
```



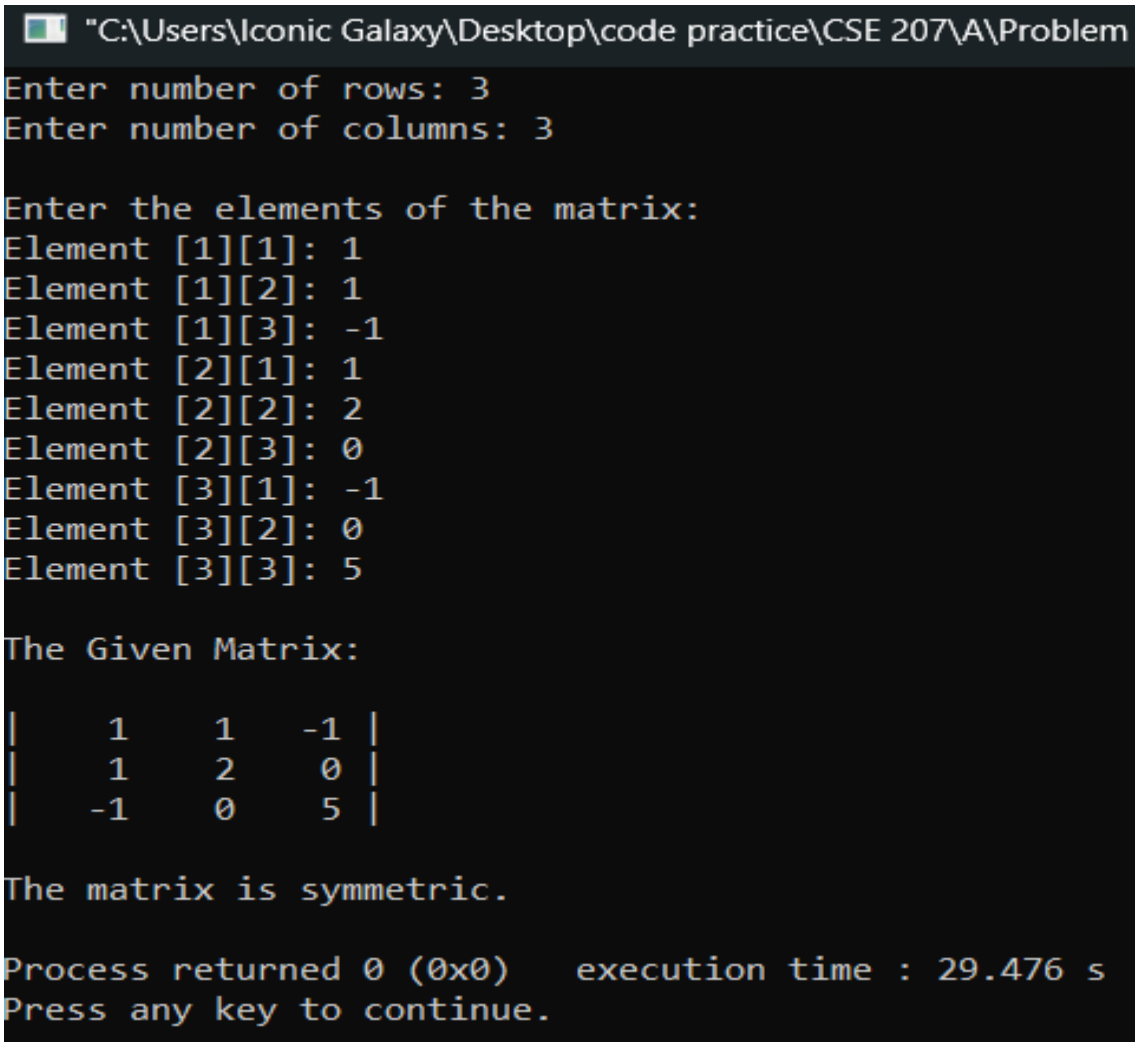
17. Write a program in C to find whether a matrix is symmetric or not.

Sample Input	Sample Output
Input array elements: 1 1 -1 1 2 0 -1 0 5	The Matrix is Symmetric.

```
#include <stdio.h>

int main()
{
   int row, column;

   printf("Enter number of rows: ");
   scanf("%d", &row);

   printf("Enter number of columns: ");
   scanf("%d", &column);

   if (row != column)
   {
      printf("Matrix is not square, so it cannot be symmetric.\n");
      return 0;
   }

   int arr[row][column];
```

```
printf("\nEnter the elements of the matrix:\n");
for (int i = 0; i < row; i++)
{
   for (int j = 0; j < column; j++)
   {
      printf("Element [%d][%d]: ", i + 1, j + 1);
      scanf("%d", &arr[i][j]);
   }
}

printf("\nThe Given Matrix:\n\n");
for (int i = 0; i < row; i++)
{
   printf("| ");
   for (int j = 0; j < column; j++)
   {
      printf("%4d ", arr[i][j]);
   }
   printf("|\n");
}

int symmetric = 1;
for (int i = 0; i < row; i++)
{
   for (int j = 0; j < column; j++)
   {
      if (arr[i][j] != arr[j][i])
      {
         symmetric = 0;
         break;
      }
   }
   if (!symmetric)
   {
      break;
   }
}

if (symmetric)
```

```
    {
        printf("\nThe matrix is symmetric.\n");
    }
    else
    {
        printf("\nThe matrix is NOT symmetric.\n");
    }
    return 0;
}
```

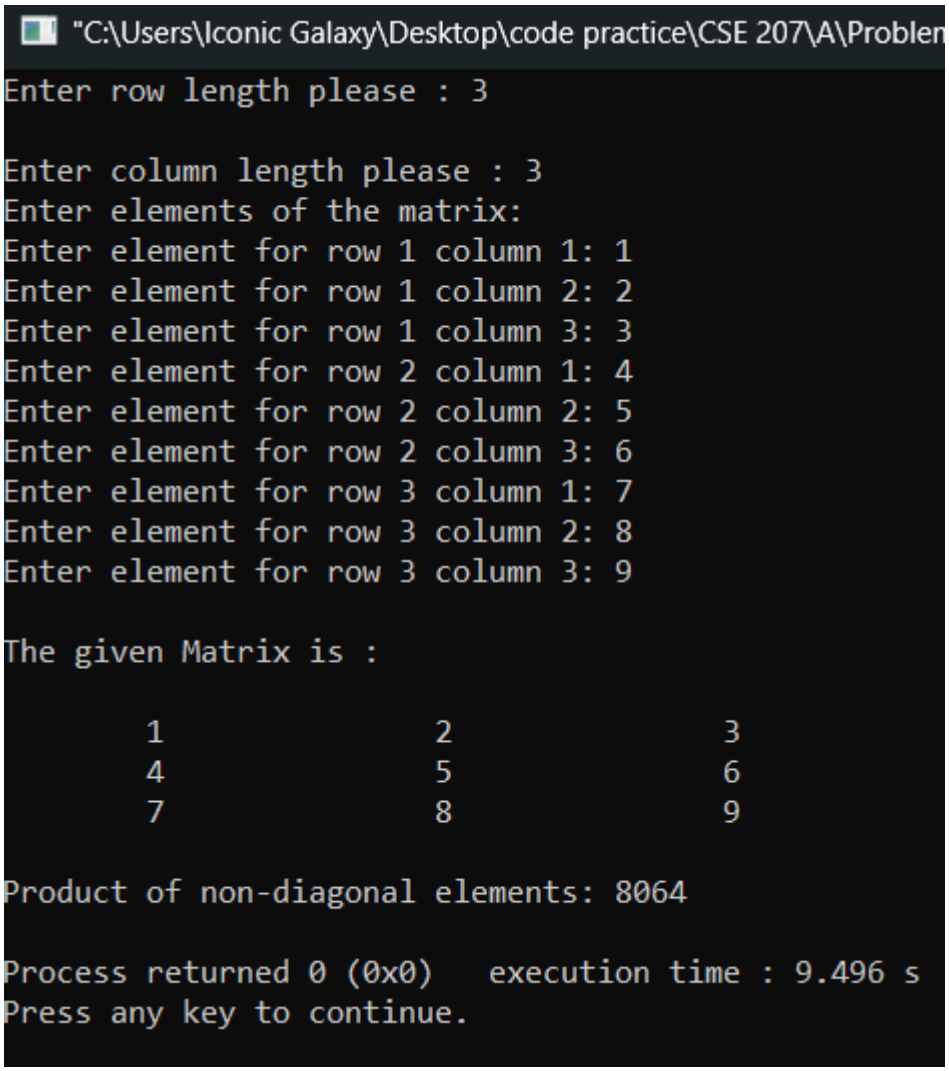


18. Write a program in C to find the product of non-Diagonal elements of a matrix.

Sample Input	Sample Output
Input array elements: 1 2 3 4 5 6 7 8 9	The product of non-diagonal elements is: 8064

```
#include<stdio.h>
int main()
{
   int row, column;

   printf("Enter row length please : ");
   scanf("%d", &row);

   printf("\nEnter column length please : ");
   scanf("%d", &column);

   int mat[row][column], k, r;
   long long int product = 1;

   printf("Enter elements of the matrix:\n");
   for(k = 0; k < row; k++)
   {
      for(r = 0; r < column; r++)
```

```
        {
            printf("Enter element for row %d column %d: ", k + 1, r + 1);
            scanf("%d", &mat[k][r]);
        }
    }

    printf("\nThe given Matrix is : \n\n");
    for(k = 0; k < row; k++)
    {
        for(r = 0; r < column; r++)
        {
            printf("\t%d\t", mat[k][r]);
        }
        printf("\n");
    }

    for(k = 0; k < row; k++)
    {
        for(r = 0; r < column; r++)
        {
            if(k != r)
            {
                product *= mat[k][r];
            }
        }
    }

    printf("\nProduct of non-diagonal elements: %lld\n", product);
    return 0;
}
```


"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\A\Problen
Enter row length please : 3
Enter column length please : 3
Enter elements of the matrix:
Enter element for row 1 column 1: 1
Enter element for row 1 column 2: 2
Enter element for row 1 column 3: 3
Enter element for row 2 column 1: 4
Enter element for row 2 column 2: 5
Enter element for row 2 column 3: 6
Enter element for row 3 column 1: 7
Enter element for row 3 column 2: 8
Enter element for row 3 column 3: 9
The given Matrix is :
1 2 3
4 5 6
7 8 9
Product of non-diagonal elements: 8064
Process returned 0 (0x0) execution time : 9.496 s
Press any key to continue.

19. Write a program in C to find the transpose of a square matrix.

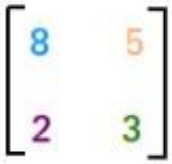
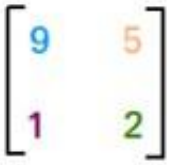
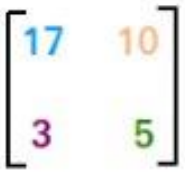
Sample Input	Sample Output
Input array elements: 1 2 3 4 5 6 7 8 9	 1 4 7 2 5 8 3 6 9

```
#include<stdio.h>
int main()
{
   int row , column ;

   printf("Enter row length please : ");
   scanf("%d" , &row);

   printf("\nEnter column length please : ");
   scanf("%d" , &column);

   int mat[row][column] , k , r  ;
   for(k = 0 ; k < row ; k++)
   {
      for(r = 0 ; r < column ; r++)
      {
         printf("Enter elements of row %d column %d\n" , k+1 , r+1);
         scanf("%d" , &mat[k][r]);

      }
   }

   printf("\nThe given Matrix is : \n\n");
   for(k = 0 ; k < row ; k++)
   {
      for(r = 0 ; r < column ; r++)
      {
         printf("\t%d   " , mat[k][r]);
```

```
        }
        printf("\n");
    }

    printf("\nTranspose matrix is : \n\n");
    for(k = 0 ; k < column ; k++)
    {
        for(r = 0 ; r < row ; r++)
        {
            printf("\t%d   " , mat[r][k]);
        }
        printf("\n");
    }
   return 0 ;
}
```

```
Enter elements of row 1 column 3
3
Enter elements of row 2 column 1
4
Enter elements of row 2 column 2
5
Enter elements of row 2 column 3
6
Enter elements of row 3 column 1
7
Enter elements of row 3 column 2
8
Enter elements of row 3 column 3
9

The given Matrix is :

        1       2       3
        4       5       6
        7       8       9

Transpose matrix is :

        1       4       7
        2       5       8
        3       6       9

Process returned 0 (0x0)   execution time : 6.
```

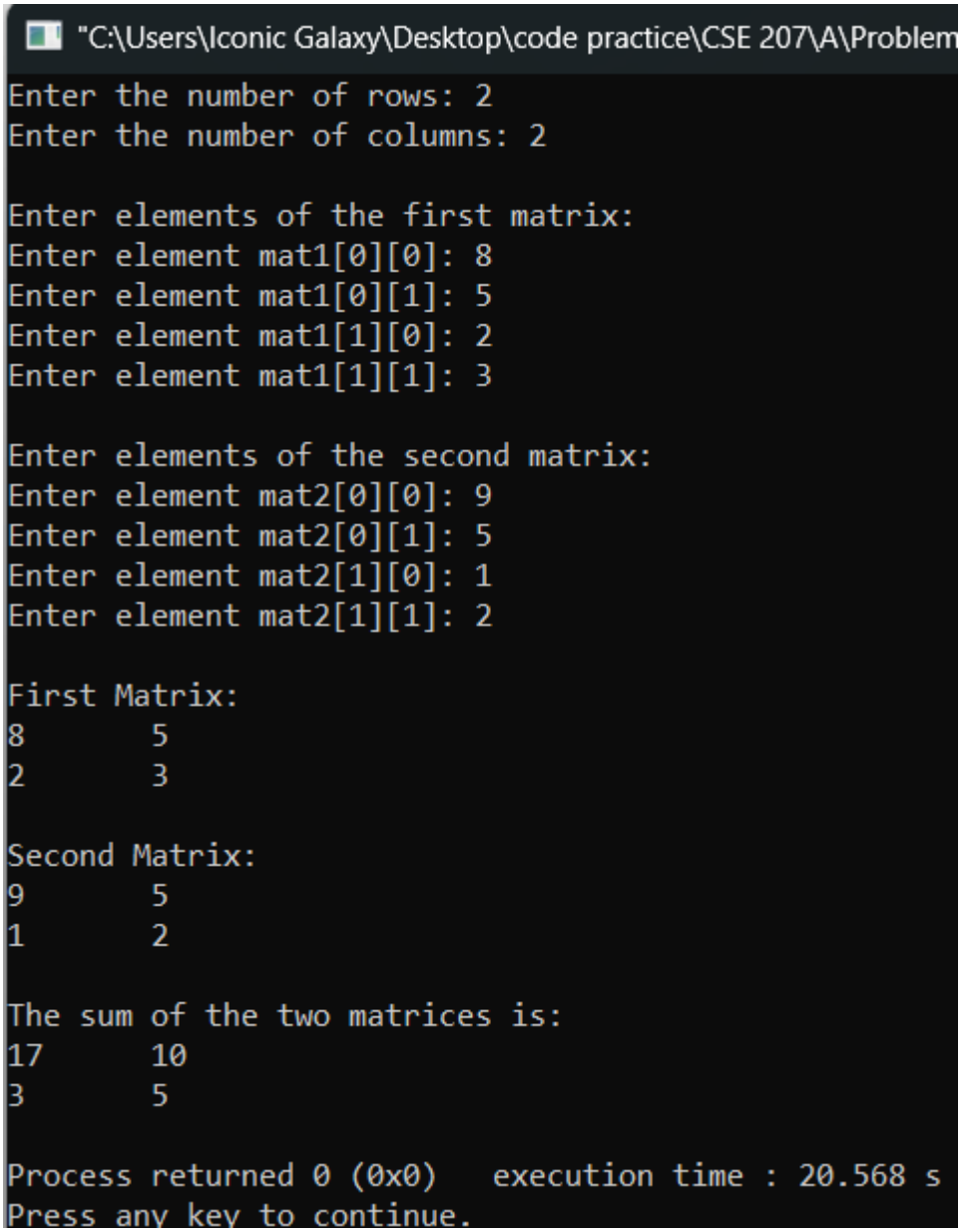
20. Write a program in C to perform matrix addition of two matrices of size *n x n*. Ensure that the program takes input for both matrices and displays the resulting matrix after addition.

Sample Input	**Sample Output**
Elements of 1st array :  Elements of 2nd array: 	Resulting array after addition: 

```
#include <stdio.h>
int main()
{
   int row, column;
   int i, j;

   printf("Enter the number of rows: ");
   scanf("%d", &row);

   printf("Enter the number of columns: ");
   scanf("%d", &column);

   int mat1[row][column], mat2[row][column], sum[row][column];

   printf("\nEnter elements of the first matrix:\n");
   for (i = 0; i < row; i++) {
      for (j = 0; j < column; j++)
      {
```

```
        printf("Enter element mat1[%d][%d]: ", i, j);
        scanf("%d", &mat1[i][j]);
    }
}
printf("\nEnter elements of the second matrix:\n");
for (i = 0; i < row; i++)
{
    for (j = 0; j < column; j++)
    {
        printf("Enter element mat2[%d][%d]: ", i, j);
        scanf("%d", &mat2[i][j]);
    }
}
printf("\nFirst Matrix:\n");
for (i = 0; i < row; i++)
{
    for (j = 0; j < column; j++)
    {
        printf("%d\t", mat1[i][j]);
    }
    printf("\n");
}
printf("\nSecond Matrix:\n");
for (i = 0; i < row; i++)
{
    for (j = 0; j < column; j++)
    {
        printf("%d\t", mat2[i][j]);
    }
    printf("\n");
}
for (i = 0; i < row; i++)
{
    for (j = 0; j < column; j++)
    {
        sum[i][j] = mat1[i][j] + mat2[i][j];
    }
}
printf("\nThe sum of the two matrices is:\n");
for (i = 0; i < row; i++)
```

```
    {
        for (j = 0; j < column; j++)
        {
            printf("%d\t", sum[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```



21. Write a program in C that takes a 2D array as user input and finds the column with the maximum sum (i.e., identify the column whose sum of elements is the highest among all columns in the array).

Sample Input	Sample Output
1 2 3 4 5 6 7 8 9	3 No. column has the maximum sum which is 18

```
#include <stdio.h>
int main()
{
   int row, col;

   printf("Enter the number of rows: ");
   scanf("%d", &row);
   printf("Enter the number of columns: ");
   scanf("%d", &col);

   int arr[row][col];
   printf("Enter the elements of the array:\n");
   for (int i = 0; i < row; i++)
   {
      for (int j = 0; j < col; j++)
      {
         scanf("%d", &arr[i][j]);
      }
   }

   int maxSum = 0;
   int maxColIndex = 0;

   for (int j = 0; j < col; j++)
   {
      int currentSum = 0;
      for (int i = 0; i < row; i++)
      {
         currentSum += arr[i][j];
```

```
        }

        if (currentSum > maxSum)
        {
            maxSum = currentSum;
            maxColIndex = j;
        }
    }
    printf("\nOriginal Matrix:\n");
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {
            printf("%d\t", arr[i][j]);
        }
        printf("\n");
    }
    printf("\nColumn with the maximum sum is column %d with a sum of %d.\n", maxColIndex+1, maxSum);
    return 0;
}
```

```
Enter the number of rows: 3
Enter the number of columns: 3
Enter the elements of the array:
1
2
3
4
5
6
7
8
9

Original Matrix:
1       2       3
4       5       6
7       8       9

Column with the maximum sum is column 3 with a sum of 18.

Process returned 0 (0x0)   execution time : 7.015 s
Press any key to continue.
```